Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

Bitcoin: un sistema de dinero electrónico entre pares

Satoshi
Nakamotosatoshin@g
mx.com
www.bitcoin.org

Resumen. Una versión de dinero electrónico puramente entre pares permitiría que los pagos en línea se enviaran directamente de una parte a otra sin pasar por una entidad financiera. Las firmas digitales aportan parte de la solución, pero las principales ventajas se pierden si sigue siendo necesario recurrir a un tercero de confianza para evitar el doble gasto. Proponemos una solución al problema del doble gasto mediante una red entre pares. La red marca temporalmente las transacciones mediante su hash en una cadena continua de prueba de trabajo basada en hash, formando un registro que no puede modificarse sin volver a realizar la prueba de trabajo. La cadena más larga no solo sirve como prueba de la secuencia de eventos atestiguados, sino también como prueba de que procede del mayor conjunto de potencia de CPU. Siempre que la mayoría de la potencia de CPU esté controlada por nodos que no cooperen para atacar la red, estos generarán la cadena más larga y superarán a los atacantes. La propia red requiere una estructura mínima. Los mensajes se transmiten según el principio de «mejor esfuerzo», y los nodos pueden abandonar y volver a unirse a la red a su antojo, aceptando la cadena de prueba de trabajo más larga como prueba de lo que ocurrió mientras estuvieron ausentes.

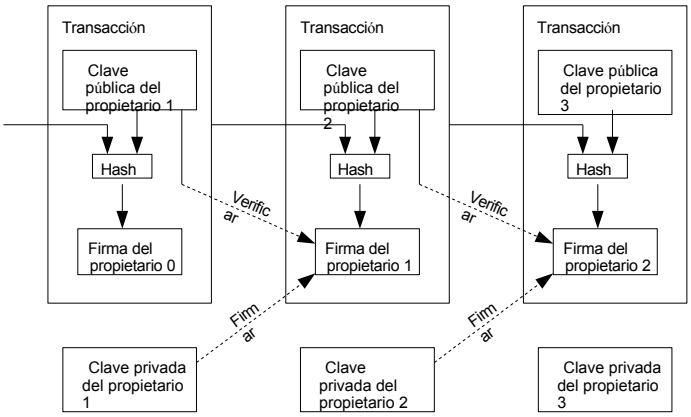
1. Introducción

El comercio en Internet ha llegado a depender casi exclusivamente de las instituciones financieras, que actúan como terceros de confianza para procesar los pagos electrónicos. Aunque el sistema funciona bastante bien para la mayoría de las transacciones, sigue adoleciendo de las debilidades inherentes al modelo basado en la confianza. Las transacciones completamente irreversibles no son realmente posibles, ya que las instituciones financieras no pueden evitar mediar en las disputas. El coste de la mediación aumenta los costes de transacción, lo que limita el importe mínimo práctico de las transacciones y elimina la posibilidad de realizar pequeñas transacciones ocasionales; además, existe un coste más amplio derivado de la pérdida de la capacidad de realizar pagos irreversibles por servicios irreversibles. Ante la posibilidad de reversión, la necesidad de confianza se extiende. Los comerciantes deben desconfiar de sus clientes, lo que les obliga a solicitarles más información de la que, de otro modo, necesitarían. Se acepta que un cierto porcentaje de fraude es inevitable. Estos costes y las incertidumbres en los pagos pueden evitarse en persona utilizando moneda física, pero no existe ningún mecanismo para realizar pagos a través de un canal de comunicación sin una parte de confianza.

Lo que se necesita es un sistema de pago electrónico basado en pruebas criptográficas en lugar de en la confianza, que permita a dos partes dispuestas a ello realizar transacciones directamente entre sí sin necesidad de un tercero de confianza. Las transacciones que resulten computacionalmente inviables de revertir protegerían a los vendedores del fraude, y se podrían implementar fácilmente mecanismos de depósito en garantía rutinarios para proteger a los compradores. En este artículo, proponemos una solución al problema del doble gasto mediante un servidor de marcas de tiempo distribuido entre pares para generar una prueba computacional del orden cronológico de las transacciones. El sistema es seguro siempre que los nodos honestos controlen colectivamente más potencia de CPU que cualquier grupo cooperativo de nodos atacantes.

2. Transacciones

Definimos una moneda electrónica como una cadena de firmas digitales. Cada propietario transfiere la moneda al siguiente firmando digitalmente un hash de la transacción anterior y la clave pública del siguiente propietario, y añadiendo estos datos al final de la moneda. Un beneficiario puede verificar las firmas para comprobar la cadena de propiedad.

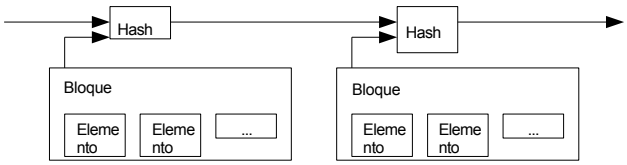


El problema, por supuesto, es que el beneficiario no puede verificar que ninguno de los propietarios haya realizado un doble gasto con la moneda. Una solución habitual consiste en introducir una autoridad central de confianza, o «casa de la moneda», que compruebe cada transacción para detectar posibles dobles gastos. Tras cada transacción, la moneda debe devolverse a la casa de la moneda para que se emita una nueva, y solo se confía en que las monedas emitidas directamente por la casa de la moneda no hayan sido objeto de un doble gasto. El problema de esta solución es que el destino de todo el sistema monetario depende de la empresa que gestiona la casa de la moneda, ya que todas las transacciones deben pasar por ella, igual que en un banco.

Necesitamos una forma de que el beneficiario sepa que los propietarios anteriores no han firmado ninguna transacción anterior. A efectos de nuestro sistema, la transacción más antigua es la que cuenta, por lo que no nos preocupan los intentos posteriores de doble gasto. La única forma de confirmar la ausencia de una transacción es conocer todas las transacciones. En el modelo basado en la casa de moneda, esta tenía constancia de todas las transacciones y decidía cuál había llegado primero. Para lograrlo sin una parte de confianza, las transacciones deben anunciarse públicamente [1], y necesitamos un sistema para que los participantes acuerden un único historial del orden en que se recibieron. El beneficiario necesita una prueba de que, en el momento de cada transacción, la mayoría de los nodos coincidían en que era la primera recibida.

3. Servidor de marcas de tiempo

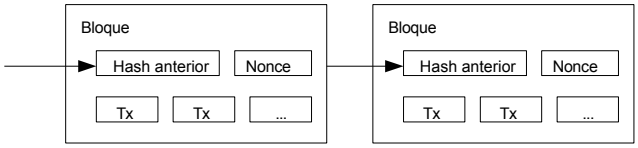
La solución que proponemos parte de un servidor de marcas de tiempo. Un servidor de marcas de tiempo funciona calculando un hash de un bloque de elementos a los que se va a aplicar la marca de tiempo y publicando ampliamente dicho hash, por ejemplo, en un periódico o en una publicación de Usenet [2-5]. La marca de tiempo demuestra que, obviamente, los datos debían existir en ese momento para poder incluirse en el hash. Cada marca de tiempo incluye la marca de tiempo anterior en su hash, formando una cadena, en la que cada marca de tiempo adicional refuerza las anteriores.



4. Prueba de trabajo

Para implementar un servidor de marcas de tiempo distribuido en una red peer-to-peer, tendremos que utilizar un sistema de prueba de trabajo similar al Hashcash de Adam Back [6], en lugar de publicaciones en periódicos o en Usenet. La prueba de trabajo consiste en buscar un valor que, al ser sometido a un algoritmo hash —como el SHA-256—, genere un hash que comience con una serie de bits cero. El esfuerzo medio requerido es exponencial en función del número de bits cero necesarios y puede verificarse ejecutando un único hash.

Para nuestra red de marcas de tiempo, implementamos la prueba de trabajo incrementando un nonce en el bloque hasta que se encuentre un valor que proporcione al hash del bloque los bits de cero requeridos. Una vez que se ha invertido el esfuerzo de la CPU para que cumpla con la prueba de trabajo, el bloque no puede modificarse sin volver a realizar el trabajo. Dado que los bloques posteriores se encadenan a partir de él, el trabajo necesario para modificar el bloque implicaría volver a generar todos los bloques posteriores.



La prueba de trabajo también resuelve el problema de determinar la representación en la toma de decisiones por mayoría. Si la mayoría se basara en el principio de «una dirección IP, un voto», cualquiera capaz de asignarse muchas direcciones IP podría subvertirla. La prueba de trabajo es, en esencia, «una CPU, un voto». La decisión mayoritaria viene representada por la cadena más larga, que es aquella en la que se ha invertido el mayor esfuerzo de prueba de trabajo. Si la mayoría de la potencia de CPU está controlada por nodos honestos, la cadena honesta crecerá más rápido y superará a cualquier cadena competidora. Para modificar un bloque anterior, un atacante tendría que volver a realizar la prueba de trabajo de ese bloque y de todos los bloques posteriores, y luego ponerse al día y superar el trabajo de los nodos honestos. Más adelante demostraremos que la probabilidad de que un atacante más lento se ponga al día disminuye exponencialmente a medida que se añaden bloques posteriores.

Para compensar el aumento de la velocidad del hardware y la variación del interés en ejecutar nodos a lo largo del tiempo, la dificultad de la prueba de trabajo se determina mediante una media móvil que tiene como objetivo un número medio de bloques por hora. Si se generan demasiado rápido, la dificultad aumenta.

5. Red

Los pasos para poner en marcha la red son los siguientes:

1) Las nuevas transacciones se transmiten a todos los nodos.
2) Cada nodo recopila las nuevas transacciones en un bloque.
3) Cada nodo se dedica a buscar una prueba de trabajo difícil para su bloque.
4) Cuando un nodo encuentra una prueba de trabajo, transmite el bloque a todos los nodos.
5) Los nodos aceptan el bloque solo si todas las transacciones que contiene son válidas y no se han gastado ya.
6) Los nodos expresan su aceptación del bloque trabajando en la creación del siguiente bloque de la cadena, utilizando el hash del bloque aceptado como hash anterior.

Los nodos siempre consideran que la cadena más larga es la correcta y seguirán trabajando para prolongarla. Si dos nodos transmiten simultáneamente versiones diferentes del siguiente bloque, es posible que algunos nodos reciban primero una u otra. En ese caso, trabajan en la primera que hayan recibido, pero guardan la otra rama por si acaso esta se alarga. El empate se resolverá cuando se encuentre la siguiente prueba de trabajo y una de las ramas se haga más larga; los nodos que estuvieran trabajando en la otra rama pasarán entonces a la más larga.

Las nuevas transmisiones de transacciones no tienen por qué llegar necesariamente a todos los nodos. Siempre que lleguen a muchos nodos, acabarán formando parte de un bloque en poco tiempo. Las transmisiones de bloques también son tolerantes a la pérdida de mensajes. Si un nodo no recibe un bloque, lo solicitará cuando reciba el siguiente bloque y se dé cuenta de que se ha perdido uno.

6. Incentivo

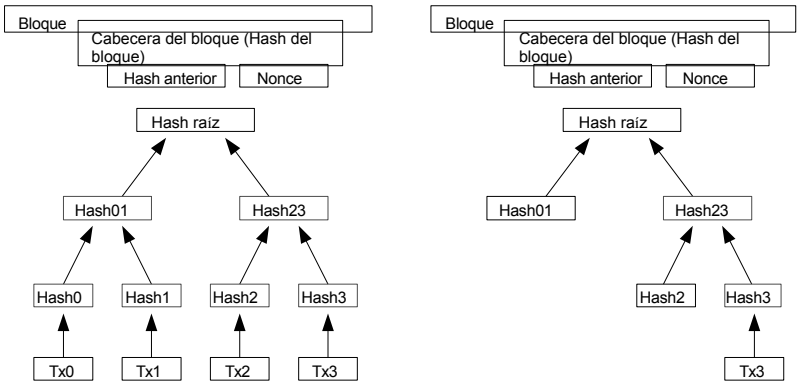
Por convención, la primera transacción de un bloque es una transacción especial que crea una nueva moneda propiedad del creador del bloque. Esto supone un incentivo para que los nodos respalden la red y proporciona una forma de distribuir inicialmente las monedas en circulación, ya que no existe una autoridad central que las emita. La incorporación constante de una cantidad fija de nuevas monedas es análoga a la de los mineros de oro que invierten recursos para añadir oro a la circulación. En nuestro caso, lo que se invierte es tiempo de CPU y electricidad.

El incentivo también puede financiarse con comisiones por transacción. Si el valor de salida de una transacción es inferior a su valor de entrada, la diferencia constituye una comisión por transacción que se suma al valor del incentivo del bloque que contiene dicha transacción. Una vez que haya entrado en circulación un número predeterminado de monedas, el incentivo puede pasar a financiarse íntegramente con las comisiones por transacción y estar completamente libre de inflación.

El incentivo puede contribuir a que los nodos se mantengan honestos. Si un atacante codicioso es capaz de reunir más potencia de CPU que todos los nodos honestos, tendría que elegir entre utilizarla para estafar a la gente recuperando sus pagos, o utilizarla para generar nuevas monedas. Le resultaría más rentable seguir las reglas —unas reglas que le favorecen con más monedas nuevas que todos los demás juntos— que socavar el sistema y la validez de su propia riqueza.

7. Recuperación de espacio en disco

Una vez que la última transacción de una moneda queda enterrada bajo suficientes bloques, las transacciones gastadas anteriores a ella pueden descartarse para ahorrar espacio en disco. Para facilitar esto sin romper el hash del bloque, las transacciones se someten a un proceso de hash en un árbol de Merkle [7][2][5], incluyéndose únicamente la raíz en el hash del bloque. Los bloques antiguos pueden entonces compactarse eliminando ramas del árbol. No es necesario almacenar los hashes interiores.



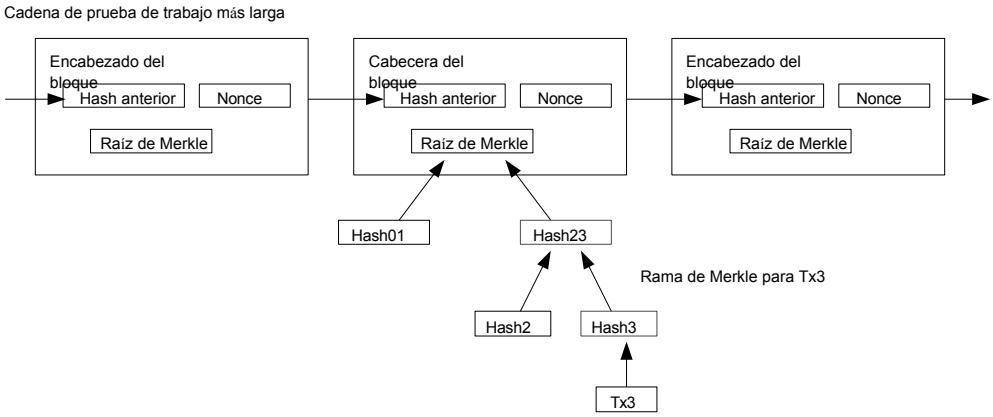
Transacciones con hash en un árbol de Merkle

Tras eliminar las transacciones Tx0-2 del bloque

Una cabecera de bloque sin transacciones tendría un tamaño aproximado de 80 bytes. Si suponemos que los bloques se generan cada 10 minutos, 80 bytes * 6 * 24 * 365 = 4,2 MB al año. Teniendo en cuenta que, en 2008, los sistemas informáticos solían venderse con 2 GB de RAM, y que la Ley de Moore prevé un crecimiento actual de
1,2 GB al año, el almacenamiento no debería suponer un problema, incluso si las cabeceras de los bloques tuvieran que mantenerse en memoria.

8. Verificación simplificada de pagos

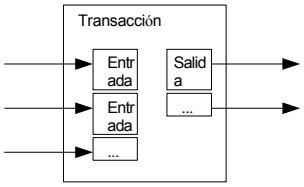
Es posible verificar los pagos sin ejecutar un nodo completo de la red. Un usuario solo necesita conservar una copia de las cabeceras de bloque de la cadena de prueba de trabajo más larga, que puede obtener consultando a los nodos de la red hasta que esté convencido de que tiene la cadena más larga, y obtener la rama de Merkle que vincula la transacción con el bloque en el que está marcada con la marca de tiempo. No puede comprobar la transacción por sí mismo, pero al vincularla a un punto de la cadena, puede ver que un nodo de la red la ha aceptado, y los bloques añadidos después de ella confirman además que la red la ha aceptado.



Por lo tanto, la verificación es fiable siempre que los nodos honestos controlen la red, pero resulta más vulnerable si un atacante logra dominar la red. Aunque los nodos de la red pueden verificar las transacciones por sí mismos, el método simplificado puede ser engañado por transacciones falsificadas por un atacante mientras este siga dominando la red. Una estrategia para protegerse contra esto consistiría en aceptar alertas de los nodos de la red cuando detecten un bloque no válido, lo que haría que el software del usuario descargara el bloque completo y las transacciones señaladas para confirmar la inconsistencia. Es probable que las empresas que reciben pagos con frecuencia sigan queriendo ejecutar sus propios nodos para disfrutar de una seguridad más independiente y una verificación más rápida.

9. Combinación y división del valor

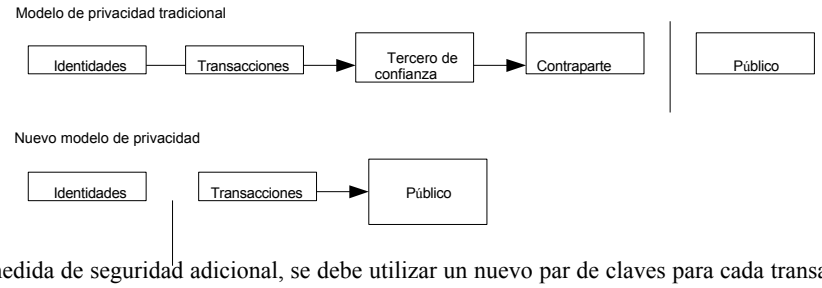
Aunque sería posible gestionar las monedas de forma individual, resultaría poco práctico realizar una transacción separada por cada céntimo de una transferencia. Para permitir que el valor se divida y se combine, las transacciones contienen múltiples entradas y salidas. Normalmente habrá una única entrada procedente de una transacción anterior de mayor cuantía o múltiples entradas que combinen cantidades más pequeñas, y como máximo dos salidas: una para el pago y otra para devolver el cambio, si lo hubiera, al remitente.



Cabe señalar que el «fan-out» —es decir, cuando una transacción depende de varias transacciones, y estas, a su vez, dependen de muchas más— no supone ningún problema en este caso. Nunca es necesario extraer una copia completa e independiente del historial de una transacción.

10. Privacidad

El modelo bancario tradicional logra un nivel de privacidad al limitar el acceso a la información a las partes implicadas y al tercero de confianza. La necesidad de anunciar públicamente todas las transacciones impide este método, pero la privacidad puede mantenerse interrumpiendo el flujo de información en otro punto: manteniendo el anonimato de las claves públicas. El público puede ver que alguien está enviando una cantidad a otra persona, pero sin información que vincule la transacción con nadie. Esto es similar al nivel de información que divulgan las bolsas de valores, donde se hacen públicos la hora y el volumen de las operaciones individuales —el «registro de operaciones»—, pero sin revelar quiénes eran las partes.



Como medida de seguridad adicional, se debe utilizar un nuevo par de claves para cada transacción, a fin de evitar que se puedan vincular a un mismo propietario. Sin embargo, en el caso de las transacciones con múltiples entradas, es inevitable que se produzca cierta vinculación, ya que estas revelan necesariamente que sus entradas pertenecían al mismo propietario. El riesgo radica en que, si se revela la identidad del propietario de una clave, dicha vinculación podría revelar otras transacciones que pertenecieran al mismo propietario.

11. Cálculos

Consideramos el escenario en el que un atacante intenta generar una cadena alternativa más rápido que la cadena honesta. Aunque lo consiga, esto no expone al sistema a cambios arbitrarios, como crear valor de la nada o apropiarse de dinero que nunca le perteneció. Los nodos no aceptarán una transacción inválida como pago, y los nodos honestos nunca aceptarán un bloque que las contenga. Un atacante solo puede intentar modificar una de sus propias transacciones para recuperar el dinero que acaba de gastar.

La carrera entre la cadena honesta y la cadena del atacante puede caracterizarse como un paseo aleatorio binomial. El evento de éxito consiste en que la cadena honesta se amplíe en un bloque, aumentando su ventaja en +1, y el evento de fracaso consiste en que la cadena del atacante se amplíe en un bloque, reduciendo la diferencia en -1.

La probabilidad de que un atacante recupere el déficit partiendo de una situación determinada es análoga al problema de la «ruina del jugador». Supongamos que un jugador con crédito ilimitado parte con un déficit y realiza un número potencialmente infinito de intentos para intentar alcanzar el umbral de rentabilidad. Podemos calcular la probabilidad de que llegue alguna vez al umbral de rentabilidad, o de que un atacante alcance alguna vez a la cadena honesta, de la siguiente manera [8]:

p = probabilidad de que un nodo honesto encuentre el siguiente bloque
q = probabilidad de que el atacante encuentre el siguiente bloque
q_z = probabilidad de que el atacante consiga alguna vez recuperar el retraso de z bloques

$$q_z = \begin{cases} 1 & si\ p \leq q \\ (q/p)^z & si\ p > q \end{cases}$$

Dada nuestra hipótesis de que $p > q$, la probabilidad disminuye exponencialmente a medida que aumenta el número de bloques que el atacante tiene que recuperar. Con las probabilidades en su contra, si no da un salto de suerte al principio, sus posibilidades se vuelven ínfimas a medida que se queda más atrás.

Consideremos ahora cuánto tiempo debe esperar el destinatario de una nueva transacción antes de tener la certeza suficiente de que el remitente no puede modificarla. Supongamos que el remitente es un atacante que quiere hacer creer al destinatario que le ha pagado durante un tiempo, para luego, una vez transcurrido cierto tiempo, cambiar la transacción para que el pago se le devuelva a sí mismo. El destinatario recibirá una alerta cuando eso ocurra, pero el remitente espera que ya sea demasiado tarde.

El destinatario genera un nuevo par de claves y entrega la clave pública al remitente poco antes de firmar. Esto impide que el remitente prepare una cadena de bloques por adelantado, trabajando en ella de forma continua hasta que tenga la suerte de adelantarse lo suficiente y, a continuación, ejecute la transacción en ese momento. Una vez enviada la transacción, el remitente deshonesto comienza a trabajar en secreto en una cadena paralela que contiene una versión alternativa de su transacción.

El destinatario espera hasta que la transacción se haya añadido a un bloque y se hayan encadenado z bloques a continuación. No conoce el progreso exacto que ha logrado el atacante, pero suponiendo que los bloques honestos hayan tardado el tiempo medio esperado por bloque, el progreso potencial del atacante seguirá una distribución de Poisson con un valor esperado:

$$\lambda = z\frac{q}{p}$$

Para obtener la probabilidad de que el atacante aún pueda alcanzar al receptor en este momento, multiplicamos la densidad de Poisson correspondiente a cada nivel de avance que podría haber logrado por la probabilidad de que pueda alcanzarlo a partir de ese punto:

$$\sum_{k=0}^{\infty}\frac{\lambda^k e^{-\lambda}}{k!}\begin{cases}(q/p)^{(z-k)} & si\ k \le z\\ 1 & si\ k>z\end{cases}$$

Reorganizando para evitar sumar la cola infinita de la distribución...

$$1-\sum_{k=0}^{z}\frac{\lambda^k e^{-\lambda}}{k!}\left(1-(q/p)^{(z-k)}\right)$$

Convirtiendo a código C...

```
#include <math.h>
double ProbabilidadDeÉxitoDelAtacante(double q, int z)
{
     double p = 1.0 - q;
     double lambda = z * (q / p); double
     suma = 1.0;
     int i, k;
     for (k = 0; k <= z; k++)
     {
          double poisson = exp(-lambda); for
          (i = 1; i <= k; i++)
               poisson *= lambda / i;
          sum -= poisson * (1 - pow(q / p, z - k));
     }
     return sum;
}
```

Al ejecutar algunos resultados, podemos observar que la probabilidad disminuye exponencialmente con z.

```
q=0,1
z = 0    P = 1,0000000
z = 1    P = 0,2045873
z = 2    P = 0,0509779
z = 3    P = 0,0131722
z = 4    P = 0,0034552
z = 5    P = 0,0009137
z = 6    P = 0,0002428
z = 7    P = 0,0000647
z = 8    P = 0,0000173
z = 9    P = 0,0000046
z = 10   P = 0,0000012

q = 0,3
z=0      P = 1,0000000
z = 5    P = 0,1773523
z = 10   P = 0,0416605
z = 15   P = 0,0101008
z = 20   P = 0,0024804
z = 25   P = 0,0006132
z = 30   P = 0,0001522
z = 35   P = 0,0000379
z = 40   P = 0,0000095
z = 45   P = 0,0000024
z = 50   P = 0,0000006
```

Resolviendo para P inferior al 0,1 %...

```
P < 0,001 q =
0,10       z =
5
q = 0,15   z = 8
q = 0,20   z = 11
q = 0,25   z = 15
q = 0,30   z = 24
q = 0,35   z = 41
q = 0,40   z = 89
q = 0,45   z = 340
```

12. Conclusión

Hemos propuesto un sistema para realizar transacciones electrónicas sin depender de la confianza. Partimos del marco habitual de las monedas basadas en firmas digitales, que proporciona un fuerte control de la propiedad, pero que resulta incompleto sin una forma de evitar el doble gasto. Para resolverlo, propusimos una red entre pares que utiliza la prueba de trabajo para registrar un historial público de transacciones que rápidamente se vuelve computacionalmente inviable de modificar para un atacante si los nodos honestos controlan la mayoría de la potencia de CPU. La red es robusta gracias a su simplicidad no estructurada. Los nodos funcionan todos a la vez con escasa coordinación. No es necesario identificarlos, ya que los mensajes no se envían a ningún lugar concreto y solo deben entregarse según el principio de «mejor esfuerzo». Los nodos pueden abandonar y volver a unirse a la red a su antojo, aceptando la cadena de prueba de trabajo como prueba de lo que ocurrió mientras estuvieron ausentes. Votan con su potencia de CPU, expresando su aceptación de los bloques válidos al trabajar en su ampliación y rechazando los bloques inválidos al negarse a trabajar en ellos. Cualquier norma o incentivo necesario puede aplicarse mediante este mecanismo de consenso.

Referencias

[1] W. Dai, «b-money», http://www.weidai.com/bmoney.txt, 1998.

[2] H. Massias, X.S. Avila y J.-J. Quisquater, «Design of a secure timestamping service with minimal trust requirements», en 20.º *Simposio sobre Teoría de la Información en el Benelux*, mayo de 1999.

[3] S. Haber, W.S. Stornetta, «How to time-stamp a digital document», en *Journal of Cryptology*, vol. 3, n.º 2, pp. 99-111, 1991.

[4] D. Bayer, S. Haber y W. S. Stornetta, «Mejora de la eficiencia y la fiabilidad del sellado de tiempo digital», en *Sequences II: Methods in Communication, Security and Computer Science*, pp. 329-334, 1993.

[5] S. Haber, W. S. Stornetta, «Nombres seguros para cadenas de bits», en *Actas de la 4.ª Conferencia ACM sobre Seguridad Informática y de las Comunicaciones*, páginas 28-35, abril de 1997.

[6] A. Back, «Hashcash: una contramedida contra los ataques de denegación de servicio», http://www.hashcash.org/papers/hashcash.pdf, 2002.

[7] R.C. Merkle, «Protocols for public key cryptosystems», en *Actas del Simposio de 1980 sobre Seguridad y Privacidad*, IEEE Computer Society, páginas 122-133, abril de 1980.

[8] W. Feller, «Una introducción a la teoría de la probabilidad y sus aplicaciones», 1957.